

Aplicaciones Jakarta EE

Resumen de soluciones y explicación del código

Ejercicios 1, 1.2, 2 y 4

Entidad gestionada: **TrabajadorSalud**

Proyecto Maven multi-módulo: `tse2026:ej1`

Servidor: WildFly 41.0.0.Final • Jakarta EE 11

Índice

1. Arquitectura general del proyecto
2. Ejercicio 1 — Aplicaciones Jakarta EE y Session Beans
 - 1.1 La entidad `TrabajadorSalud`
 - 1.2 Capa de Acceso a Datos — Singleton Session Bean
 - 1.3 Capa de Negocio — Stateless Session Bean
 - 1.4 Capa de Presentación Web — Servlet + JSP
 - 1.5 Capa de Presentación — Aplicación de consola
 - 1.6 Errores detectados y corregidos
 - 1.7 Prueba unitaria sin contenedor
3. Ejercicio 1.2 — Despliegue en WildFly local con Docker
4. Ejercicio 2 — Despliegue en la Nube (PaaS / OpenShift)
5. Ejercicio 4 — Capa de presentación con JSF y AJAX
6. Cómo ejecutar todo

1. Arquitectura general del proyecto

La solución es un proyecto **Maven multi-módulo** que separa las tres capas clásicas de una aplicación empresarial y empaqueta todo en un **EAR** desplegable en WildFly.

Módulo	Empaque	Capa / rol
ej1-api	jar	Interfaz remota de negocio (contrato compartido con el cliente Java)
ejb	ejb	Capa de Negocio (Stateless) + Capa de Datos (Singleton) + entidad
web	war	Capa de Presentación Web: Servlet+JSP y JSF (Ej. 4)
ear	ear	Empaqueta <code>ejb</code> + <code>web</code> ; incluye <code>ej1-api.jar</code> en <code>lib/</code>
ej1-console	jar	Capa de Presentación como aplicación Java de consola (EJB remoto)

Flujo de una petición

```
Navegador / Consola
↓
Capa Presentación  (Servlet+JSP | JSF | app consola)
  ↓ interfaz LOCAL (web) / interfaz REMOTA (consola)
Capa Negocio      @Stateless TrabajadorSaludService ← valida la regla de negocio
  ↓ interfaz LOCAL
Capa Datos         @Singleton TrabajadorSaludDAO     ← almacena (List en memoria)
```

Nota sobre la persistencia: en este primer prototipo la capa de datos guarda los registros en una `List` en memoria dentro del Singleton. No se usa base de datos todavía (el `persistence.xml` queda inerte porque la entidad es un POJO, no una `@Entity` JPA). Al reiniciarse el servidor la lista se vacía.

2. Ejercicio 1 — Aplicaciones Jakarta EE y Session Beans

Se pide una aplicación Jakarta EE de tres capas para gestionar una entidad, con: capa de presentación **web (Servlet/JSP)** y **Java de consola**; capa de negocio con **Stateless Session Beans** (interfaces local y remota) que controlen **al menos una regla de negocio** al dar de alta; y capa de datos con un **Singleton Session Bean** (interfaces local y remota).

2.1 La entidad `TrabajadorSalud`

`ejb/src/main/java/tse2026/ej1/entidad/TrabajadorSalud.java`

Representa a un trabajador de la salud. Cumple la consigna de tener **al menos tres atributos de tipos distintos**: texto (`String`), numérico (`Long`), fecha (`LocalDate`) y booleano (`Boolean`).

```
public class TrabajadorSalud implements Serializable {
    private Long id;           // numérico
    private String nombre;     // texto
    private String apellido;
    private String especialidad;
    private String matricula;  // texto – clave de negocio
    private LocalDate fechaIngreso; // fecha
    private Boolean activo;    // booleano
    // constructor vacío + constructor completo + getters/setters + toString()
}
```

Por qué `implements Serializable`: las instancias viajan por la interfaz remota (consola) y se guardan en beans JSF de vista, dos escenarios que exigen que el objeto sea serializable.

2.2 Capa de Acceso a Datos — Singleton Session Bean

`ejb/src/main/java/tse2026/ej1/datos/TrabajadorSaludDAO.java` (+ `...DAOLocal`, `...DAORemote`)

Interfaz local

```
@Local
public interface TrabajadorSaludDAOLocal {
    void agregar(TrabajadorSalud trabajador);
    List<TrabajadorSalud> obtenerTodos();
    TrabajadorSalud buscarPorMatricula(String matricula);
}
```

Interfaz remota

```
@Remote
public interface TrabajadorSaludDAORemote {
    void agregar(Long id, String nombre, String apellido, String especialidad,
        String matricula, String fechaIngreso, Boolean activo);
}
```

```
String listar();
String buscarRemotoPorMatricula(String matricula);
}
```

La interfaz remota trabaja con **tipos simples y devuelve texto** para que un cliente remoto no dependa de la clase entidad.

Implementación

```
@Singleton
public class TrabajadorSaludDAO
    implements TrabajadorSaludDAOLocal, TrabajadorSaludDAORemote {

    private List<TrabajadorSalud> trabajadores = new ArrayList<>();

    @Override public void agregar(TrabajadorSalud t) { trabajadores.add(t); }

    @Override public List<TrabajadorSalud> obtenerTodos() { return trabajadores; }

    @Override public TrabajadorSalud buscarPorMatricula(String matricula) {
        for (TrabajadorSalud t : trabajadores)
            if (t.getMatricula().equals(matricula)) return t;
        return null;
    }
    // ... versiones de la interfaz remota que arman/serializan a String
}
```

Explicación:

- `@Singleton` : el contenedor crea **una única instancia** compartida por toda la aplicación. Es ideal como “almacén” en memoria: todas las capas (web, JSF, consola) ven la misma lista.
- Por defecto un Singleton tiene **concurrency gestionada por el contenedor** con bloqueo de escritura, así que los accesos concurrentes quedan serializados sin código extra.
- Implementa **las dos interfaces** (`@Local` y `@Remote`), como pide la letra.

2.3 Capa de Negocio — Stateless Session Bean

ejb/src/main/java/tse2026/ej1/negocio/TrabajadorSaludService.java (+ `...ServiceLocal`, y `...ServiceRemote` en `ej1-api`)

Implementación (parte local)

```
@Stateless
public class TrabajadorSaludService
    implements TrabajadorSaludServiceLocal, TrabajadorSaludServiceRemote {

    @EJB
    private TrabajadorSaludDAOLocal dao; // inyección del Singleton por interfaz local

    @Override
    public void agregar(TrabajadorSalud trabajador) {
        // ===== REGLA DE NEGOCIO =====
        TrabajadorSalud existente = dao.buscarPorMatricula(trabajador.getMatricula());
        if (existente != null) {
            throw new ReglaNegocioException(
```

```

        "Ya existe un trabajador con la matrícula " + trabajador.getMatricula());
    }
    dao.agregar(trabajador);
}

@Override public List<TrabajadorSalud> obtenerTodos() { return dao.obtenerTodos(); }
@Override public TrabajadorSalud buscarPorMatricula(String m) { return dao.buscarPorMatricula(m); }
}
}

```

La regla de negocio y `ReglaNegocioException`

ejb/src/main/java/tse2026/ej1/negocio/ReglaNegocioException.java

```

@ApplicationException(rollback = true)
public class ReglaNegocioException extends RuntimeException {
    public ReglaNegocioException(String mensaje) { super(mensaje); }
}

```

Por qué una excepción propia con `@ApplicationException`: cuando un `@Stateless` lanza una `RuntimeException` común, el contenedor la **envuelve en `EJBException`** y hace rollback. La capa web entonces no puede distinguir “matrícula repetida” de un error real, y responde HTTP 500. Al anotar la excepción con `@ApplicationException`, el contenedor **la deja pasar tal cual** hasta la presentación, que la captura y muestra un mensaje claro (HTTP 400 en el Servlet, `FacesMessage` en JSF).

Explicación de `@Stateless`

- El contenedor mantiene un **pool** de instancias sin estado; cada llamada puede atenderla una instancia distinta. Perfecto para lógica de negocio: escala bien y es simple.
- Provee **interfaz local** (usada por la capa web, misma JVM → rápida) y **interfaz remota** (usada por la consola vía red).
- Transacción automática: cada método corre en su propia transacción contenedora (CMT).

Interfaz remota (módulo `ej1-api`)

```

@Remote
public interface TrabajadorSaludServiceRemote {
    String agregar(Long id, String nombre, String apellido, String especialidad,
        String matricula, String fechaIngreso, Boolean activo);
    String listar();
    String buscarRemotoPorMatricula(String matricula);
}

```

Vive en su **propio módulo `ej1-api`** para que la aplicación de consola dependa solo de este contrato liviano y no del jar completo de EJBs. El `ejb` depende de `ej1-api` con *scope compile*, de modo que `ej1-api.jar` viaja dentro del EAR (carpeta `lib/`) y queda en el classpath del servidor.

2.4 Capa de Presentación Web — Servlet + JSP

web/src/main/java/tse2026/ej1/web/TrabajadorSaludServlet.java • web/src/main/webapp/WEB-INF/trabajadores.jsp

```

@WebServlet("/trabajadores")
public class TrabajadorSaludServlet extends HttpServlet {

```

```

@EJB
private TrabajadorSaludServiceLocal service; // inyección de la capa de negocio (local)

protected void doGet(...) { /* accion = listar | buscar */ }
protected void doPost(...) { /* accion = agregar */ }

private void agregar(HttpServletRequest req, HttpServletResponse resp) throws IOException {
    try {
        TrabajadorSalud t = new TrabajadorSalud(
            Long.parseLong(req.getParameter("id")),
            req.getParameter("nombre"), req.getParameter("apellido"),
            req.getParameter("especialidad"), req.getParameter("matricula"),
            LocalDate.parse(req.getParameter("fechaIngreso")),
            Boolean.parseBoolean(req.getParameter("activo")));
        service.agregar(t);
        resp.sendRedirect(req.getContextPath() + "/trabajadores?accion=listar");
    } catch (ReglaNegocioException | IllegalArgumentException e) {
        resp.sendError(HttpServletResponse.SC_BAD_REQUEST, e.getMessage());
    }
}

private void listar(...) { req.setAttribute("trabajadores", service.obtenerTodos());
    req.getRequestDispatcher("/WEB-INF/trabajadores.jsp").forward(req,
resp); }
private void buscar(...) { /* service.buscarPorMatricula(...) → JSP */ }
}

```

Explicación:

- Patrón **MVC clásico**: el Servlet es el controlador, el JSP la vista, el EJB el modelo/negocio.
- `@EJB TrabajadorSaludServiceLocal` : el servlet usa la **interfaz local** porque corren en la misma JVM (dentro del EAR).
- El `catch` combina `ReglaNegocioException` (regla de negocio) e `IllegalArgumentException` (dato de formulario inválido, p. ej. id no numérico) y responde **HTTP 400** con el mensaje.
- Tras el alta hace `sendRedirect` (patrón POST-Redirect-GET) para evitar reenvíos.

2.5 Capa de Presentación — Aplicación de consola

ej1-console/src/main/java/tse2026/ej1/console/ConsolaApp.java

```

Properties env = new Properties();
env.put(Context.INITIAL_CONTEXT_FACTORY,
    "org.wildfly.naming.client.WildFlyInitialContextFactory");
env.put(Context.PROVIDER_URL, "remote+http://localhost:8080");
env.put(Context.SECURITY_PRINCIPAL, "prueba"); // usuario de aplicación en WildFly
env.put(Context.SECURITY_CREDENTIALS, "prueba");
Context ctx = new InitialContext(env);

// ejb:<app>/<módulo>/<bean>!<interfaz remota>
String jndi = "ejb:ej1/ej1-ejb/TrabajadorSaludService!" +
    TrabajadorSaludServiceRemote.class.getName();
TrabajadorSaludServiceRemote service =
    (TrabajadorSaludServiceRemote) ctx.lookup(jndi);

// menú 1) Agregar 2) Listar 3) Buscar por matrícula 4) Salir

```

Explicación:

- La consola es un proceso Java **fuera** del servidor: obtiene el EJB por **JNDI remoto** con el esquema `ejb:`.
- `WildFlyInitialContextFactory` + `remote+http://` son el mecanismo actual de WildFly para invocación remota de EJB sobre HTTP.
- Requiere un **usuario de aplicación** (`prueba/prueba` , dado de alta con `add-user`).
- En ejecución se agrega al classpath `JBOSS_HOME/bin/client/jboss-client.jar` (script `ej1-console/ejecutar-consola.bat`).

2.6 Errores detectados y corregidos

El esqueleto tenía varios problemas que impedían compilar o desplegar. Correcciones aplicadas:

#	Problema	Corrección
1	<code>TrabajadorSaludServiceRemote</code> estaba vacía (sin <code>@Remote</code> ni métodos) → no compilaba el módulo <code>ejb</code> ni el reactor.	Se completó con <code>@Remote</code> y los 3 métodos.
2	<code>ej1-api</code> sin dependencia <code>jakarta.ejb-api</code> ; <code>ejb</code> y <code>ej1-console</code> no dependían de <code>ej1-api</code> .	Dependencias agregadas en los 3 <code>pom.xml</code> .
3	La regla de negocio devolvía HTTP 500 : el <code>@Stateless</code> envuelve las <code>RuntimeException</code> en <code>EJBException</code> .	Nueva <code>ReglaNegocioException</code> con <code>@ApplicationException(rollback=true)</code> .
4	<code>ConsolaApp</code> usaba JNDI antiguo (<code>http-remoting://</code>) sin <code>InitialContextFactory</code> → <code>NoInitialContextException</code> .	<code>WildFlyInitialContextFactory</code> + <code>remote+http://</code> + credenciales.
5	En <code>deployments/</code> de WildFly convivían el EAR y los módulos sueltos (<code>.war</code> , <code>.jar</code>); el WAR suelto fallaba al no resolver el EJB de otro <i>deployment</i> .	Se despliega solo el EAR .

2.7 Prueba unitaria sin contenedor

`ejb/src/test/java/tse2026/ej1/negocio/TrabajadorSaludServiceTest.java`

```
class TrabajadorSaludServiceTest {
    private TrabajadorSaludService service;

    @BeforeEach void setUp() {
        // el Stateless se instancia con new y se le inyecta a mano el Singleton (POJO)
        service = new TrabajadorSaludService(new TrabajadorSaludDAO());
    }

    @Test void reglaDeNegocio_noPermiteMatriculaDuplicada() {
        service.agregar(nuevo("M-200"));
        assertThrows(ReglaNegocioException.class, () -> service.agregar(nuevo("M-200")));
        assertEquals(1, service.obtenerTodos().size());
    }
}
```



```
// + agregarYListar() + buscar_matriculaInexistenteDevuelveNull()  
}
```

Se agregó al `Service` un constructor *package-private* `TrabajadorSaludService(TrabajadorSaludDAOLocal)` para poder inyectar el DAO en la prueba sin levantar WildFly. Resultado: **3 tests, 0 fallos** con `mvn test`.

3. Ejercicio 1.2 — Despliegue en WildFly local con Docker

Se empaqueta el servidor + la aplicación en una imagen Docker, a partir de la imagen oficial de WildFly.

Dockerfile

Dockerfile (raíz del proyecto)

```
FROM quay.io/wildfly/wildfly:41.0.0.Final-jdk21

# 1) Usuario de aplicación para que la consola invoque el EJB remoto
USER root
RUN /opt/jboss/wildfly/bin/add-user.sh -a -u prueba -p prueba --silent

# 2) Desplegar el EAR (el deployment scanner lo toma al arrancar)
COPY ear/target/ej1.ear /opt/jboss/wildfly/standalone/deployments/

# 3) Compatibilidad con PaaS tipo OpenShift (UID aleatorio del grupo 0)
RUN chgrp -R 0 /opt/jboss/wildfly/standalone \
    && chmod -R g+rwX /opt/jboss/wildfly/standalone

USER 1000
EXPOSE 8080 9990
CMD ["/opt/jboss/wildfly/bin/standalone.sh", "-b", "0.0.0.0", "-bmanagement", "0.0.0.0"]
```

Explicación línea por línea:

- `FROM ...wildfly:41.0.0.Final-jdk21` : misma versión que el servidor local.
- `add-user.sh -a` : crea el usuario de aplicación `prueba` dentro de la imagen.
- `COPY ear/target/ej1.ear ...deployments/` : el EAR generado por `mvn install` se copia a la carpeta que WildFly escanea; al arrancar lo despliega solo.
- `chgrp 0` + `chmod g+rwX` : en OpenShift el contenedor corre con un UID aleatorio que pertenece al grupo `0` ; los archivos deben ser legibles/escribibles por ese grupo. Sirve tanto local como en la nube.
- `-b 0.0.0.0` : WildFly acepta conexiones desde fuera del contenedor.

`.dockerignore` y `docker-compose.yml`

```
# .dockerignore - enviar al daemon solo el EAR
*
!ear/target/ej1.ear
```

```
# docker-compose.yml
services:
  wildfly:
    build: .
    image: ej1-wildfly:latest
    container_name: ej1-wildfly
    ports:
```

- "8080:8080"
- "9990:9990"

Uso y verificación

```
mvn clean install          # genera ear/target/ej1.ear
docker compose up -d --build
# App: http://localhost:8080/ej1-web/trabajadores
```

Prueba (contra el contenedor)	Resultado
Log de arranque	Deployed "ej1.ear" • WildFly started in ~4 s
Alta vía web	HTTP 302 → listado
Matrícula repetida	HTTP 400 + "Ya existe un trabajador con la matrícula..."
Listar / Buscar	correcto
Consola contra el contenedor	agregar / listar / buscar OK (EJB remoto + usuario prueba)

4. Ejercicio 2 — Despliegue en la Nube (PaaS)

PaaS (Plataforma como Servicio): el proveedor gestiona sistema operativo, runtime y escalado; nosotros solo entregamos la aplicación (aquí, una imagen Docker). Se usó **Red Hat OpenShift** (Developer Sandbox, gratuito).

Estrategia: build binario dentro del cluster

No hace falta registro de imágenes externo ni Docker local: se sube el contexto (`Dockerfile` + `ej1.ear`) y **OpenShift construye la imagen** con *buildah* y la publica en su registro interno.

```
# 1) Autenticación (token del Sandbox)
oc login --token=sha256~... --server=https://api.rm2.thpm.p1.openshiftapps.com:6443

# 2) Definir un build de estrategia docker con entrada binaria
oc new-build --name ej1 --strategy docker --binary

# 3) Subir el contexto y construir en el cluster
oc start-build ej1 --from-dir . --follow

# 4) Crear Deployment + Service desde la imagen construida
oc new-app ej1
oc set resources deploy/ej1 --requests=memory=768Mi --limits=memory=1200Mi

# 5) Publicar una URL HTTPS pública (Route con TLS edge)
oc expose svc/ej1
oc patch route/ej1 -p '{"spec":{"tls":{"termination":"edge"}}}'
```

Objetos de OpenShift/Kubernetes involucrados:

- **BuildConfig** + **ImageStream**: reciben el contexto y producen/versionan la imagen.
- **Deployment**: mantiene el pod con WildFly corriendo (lo reinicia si cae).
- **Service**: nombre y IP estable interna hacia el pod (puerto 8080).
- **Route**: expone el Service en una URL pública con HTTPS.

Resultado

URL pública: `https://ej1-juanpipitecnoinfo-dev.apps.rm2.thpm.p1.openshiftapps.com/ej1-web/trabajadores`

Prueba (contra la URL de la nube)	Resultado
Pod	Running 1/1 • “Deployed ej1.ear” sin errores
Alta	HTTP 302
Regla de negocio (matrícula repetida)	HTTP 400 + mensaje
Listar / Buscar por matrícula	correcto

Es la **misma imagen** Docker del Ejercicio 1.2 la que corre en la nube: el flujo Docker → PaaS queda completo.
Re-desplegar tras cambios: `mvn install && oc start-build ej1 --from-dir . --follow`.

5. Ejercicio 4 — Capa de presentación con JSF y AJAX

Se moderniza la capa web con **Jakarta Faces (JSF)** — implementación Mojarra 4.1 sobre WildFly — conviviendo con la versión Servlet/JSP. Todas las llamadas al servidor son por **AJAX** (etiqueta `<f:ajax>`), sin recargar la página.

5.1 Backing bean

web/src/main/java/tse2026/ej1/web/jsf/TrabajadorSaludBean.java

```
@Named
@RequestScoped
public class TrabajadorSaludBean {

    @EJB
    private TrabajadorSaludServiceLocal service;    // la MISMA capa de negocio del Ej. 1

    // modelo del formulario de alta
    private Long id; private String nombre, apellido, especialidad, matricula;
    private LocalDate fechaIngreso; private boolean activo = true;
    // búsqueda
    private String matriculaBuscar;
    // resultado mostrado en la tabla
    private List<TrabajadorSalud> trabajadores;

    @PostConstruct
    public void init() { trabajadores = service.obtenerTodos(); }

    public void agregar() {
        FacesContext ctx = FacesContext.getCurrentInstance();
        try {
            service.agregar(new TrabajadorSalud(id, nombre, apellido,
                especialidad, matricula, fechaIngreso, activo));
            ctx.addMessage(null, new FacesMessage(FacesMessage.SEVERITY_INFO,
                "Trabajador agregado", "Matricula " + matricula));
            limpiarFormulario();
        } catch (ReglaNegocioException e) {
            ctx.addMessage(null, new FacesMessage(FacesMessage.SEVERITY_ERROR,
                "No se pudo agregar", e.getMessage()));
        }
        trabajadores = service.obtenerTodos();
    }

    public void buscar() {
        if (matriculaBuscar == null || matriculaBuscar.isBlank()) {
            trabajadores = service.obtenerTodos(); return;
        }
        TrabajadorSalud t = service.buscarPorMatricula(matriculaBuscar.trim());
        trabajadores = (t == null) ? List.of() : List.of(t);
    }

    public void listarTodos() { matriculaBuscar = null; trabajadores = service.obtenerTodos(); }
```

```
// getters / setters  
}
```

Explicación:

- `@Named` : expone el bean a las páginas con el nombre EL `#{trabajadorSaludBean}` .
- `@RequestScoped` : el estado real vive en el **Singleton** de la capa de datos, no en el bean. El bean solo coordina cada petición; en cada llamada AJAX se recrea, `@PostConstruct` recarga la lista y el método de acción la ajusta. Así se evita cualquier problema de serialización/pasivación.
- `@EJB TrabajadorSaludServiceLocal` : **reutiliza sin cambios** la capa de negocio del Ejercicio 1 (misma regla de negocio, misma validación).
- La `ReglaNegocioException` se traduce a un `FacesMessage` de error visible en pantalla.

5.2 Página Facelets

web/src/main/webapp/trabajadores.xhtml

```
<h:messages id="msgs" showDetail="true" infoClass="ok" errorClass="err" />  
  
<!-- ALTA -->  
<h:form id="formAlta">  
  <h:inputText id="id" value="#{trabajadorSaludBean.id}" required="true" />  
  <h:inputText id="nombre" value="#{trabajadorSaludBean.nombre}" required="true" />  
  ... apellido, especialidad, matricula ...  
  <h:inputText id="fecha" a:type="date" value="#{trabajadorSaludBean.fechaIngreso}">  
    <f:convertDateTime type="localDate" pattern="yyyy-MM-dd" />  
  </h:inputText>  
  <h:selectBooleanCheckbox id="activo" value="#{trabajadorSaludBean.activo}" />  
  
  <h:commandButton value="Agregar trabajador" action="#{trabajadorSaludBean.agregar}">  
    <f:ajax execute="@form" render="@form :msgs :resultado" />  
  </h:commandButton>  
</h:form>  
  
<!-- BUSQUEDA (botón + búsqueda en vivo) -->  
<h:form id="formBuscar">  
  <h:inputText id="q" value="#{trabajadorSaludBean.matriculaBuscar}">  
    <f:ajax event="keyup" delay="300" listener="#{trabajadorSaludBean.buscar}" render=":resultado" />  
  </h:inputText>  
  <h:commandButton value="Buscar" action="#{trabajadorSaludBean.buscar}">  
    <f:ajax execute="@form" render=":resultado" /></h:commandButton>  
  <h:commandButton value="Mostrar todos" action="#{trabajadorSaludBean.listarTodos}">  
    <f:ajax execute="@form" render="@form :resultado" /></h:commandButton>  
</h:form>  
  
<!-- LISTADO -->  
<h:panelGroup id="resultado" layout="block">  
  <h:dataTable value="#{trabajadorSaludBean.trabajadores}" var="t"  
    rendered="#{not empty trabajadorSaludBean.trabajadores}">  
    <h:column><f:facet name="header">Matricula</f:facet>#{t.matricula}</h:column>  
    ... resto de columnas ...  
    <h:column><f:facet name="header">Activo</f:facet>  
      <h:outputText value="Si" rendered="#{t.activo}" />  
      <h:outputText value="No" rendered="#{not t.activo}" />  
    </h:column>
```

```

</h:dataTable>
<h:outputText value="No hay trabajadores para mostrar."
               rendered="#{empty trabajadorSaludBean.trabajadores}" />
</h:panelGroup>

```

Cómo funciona el AJAX (criterios de usabilidad):

- `<f:ajax execute="@form">` : al pulsar el botón se envían por AJAX solo los campos del formulario.
- `render="@form :msgs :resultado"` : el servidor devuelve **solo esos fragmentos** de HTML y JSF los reemplaza en la página — sin recarga completa.
- `event="keyup" delay="300"` : búsqueda **en vivo** mientras se escribe, con 300 ms de espera para no saturar el servidor.
- `required="true"` + `<h:messages>` : validación del lado servidor con mensajes inmediatos.
- `<f:convertDateTime type="localDate">` : convierte el texto `yyyy-MM-dd` del input HTML5 `type="date"` a `LocalDate`.

5.3 Configuración

web/src/main/webapp/WEB-INF/web.xml • WEB-INF/faces-config.xml (ya existía) • WEB-INF/beans.xml (ya existía)

```

<context-param>
  <param-name>jakarta.faces.PROJECT_STAGE</param-name>
  <param-value>Development</param-value>  <!-- mejores mensajes de error -->
</context-param>
<welcome-file-list>
  <welcome-file>trabajadores.xhtml</welcome-file>
</welcome-file-list>

```

La `FacesServlet` se activa automáticamente por la presencia de `faces-config.xml` y queda mapeada a `*.xhtml`. `beans.xml` habilita CDI.

5.4 Verificación

Acción	Resultado (sin recarga de página)
Alta AJAX	mensaje “Trabajador agregado”, fila nueva en la tabla, formulario limpio
Alta con matrícula repetida	“Ya existe un trabajador con la matrícula...”, tabla intacta, campos conservados
Buscar (botón)	la tabla se filtra
Búsqueda en vivo (keyup)	filtra mientras se escribe
Mostrar todos	vuelve el listado completo

URL: `http://localhost:8080/ej1-web/trabajadores.xhtml` (la versión Servlet/JSP sigue en `/ej1-web/trabajadores`).

6. Cómo ejecutar todo

Compilar y probar

```
cd ej1
mvn clean install          # 7 módulos + 3 tests unitarios → BUILD SUCCESS
```

A) WildFly local (standalone)

```
& "$JBOSS_HOME/bin/standalone.bat"
"$JBOSS_HOME/bin/jboss-cli.bat" --connect --command="deploy ../ear/target/ej1.ear --force"
# Web JSP : http://localhost:8080/ej1-web/trabajadores
# Web JSF : http://localhost:8080/ej1-web/trabajadores.xhtml
# Consola : ej1-console/ejecutar-consola.bat
```

B) WildFly en Docker (Ej. 1.2)

```
docker compose up -d --build
# http://localhost:8080/ej1-web/trabajadores.xhtml
docker compose down      # detener
```

C) OpenShift / PaaS (Ej. 2)

```
oc login --token=... --server=...
bash openshift/desplegar.sh
# https://ej1-<usuario>-dev.<cluster>.openshiftapps.com/ej1-web/trabajadores
```

Aviso: el contenedor Docker y el WildFly standalone usan los mismos puertos 8080/9990; ejecutar solo uno a la vez.

Resumen de estado

Ejercicio	Entregable	Estado
1	App Jakarta EE 3 capas (Singleton + Stateless + Servlet/JSP + consola), regla de negocio	Completo y probado
1.2	Imagen Docker de WildFly con el EAR desplegado	Completo y probado
2	Misma imagen desplegada en OpenShift (PaaS) con URL pública	Completo y probado
4	Capa web con JSF + AJAX (alta, listar, buscar, búsqueda en vivo)	Completo y probado